

Analyse numérique et calcul scientifique - 1

Notes de Cours Magistraux

Introduction

Ces notes de cours constituent un condensé de plusieurs ouvrages de référence, révisés et adaptés pour offrir une présentation cohérente, claire et ciblée des notions essentielles. Elles ont été spécialement élaborées pour les étudiants du Master Modélisation et Simulation en Mécanique de l'UPEC-UGE.

Le document a vocation à être **évolutif** : il sera régulièrement enrichi et corrigé afin de refléter les retours des étudiants (n'hésitez pas à me faire part de vos commentaires par email), les avancées de la discipline et les besoins pédagogiques du parcours.

Des aspects pratiques d'implémentation en Python seront abordés lors des travaux pratiques, disponibles à l'adresse suivante : https://github.com/felipefr/numerical_analysis/

1 Organisation 2025-2026

- CMs :
 1. 23/09 (3h) : Rappels mathématiques, concepts généraux, erreurs numériques, régression, introduction à Python.
 2. 06/10 (1,5h) : Interpolation.
 3. 24/11 (1,5h) : Intégration numérique.
 4. 01/12 (3h) : Résolution numérique des équations différentielles ordinaires.
- TPs (GPA) : 29/09 (3h), 13/10 (3h), 02/12 (3h), 09/12 (3h).
- TPs (GPB) : 06/10 (3h), 20/10 (3h), 02/12 (3h), 09/12 (3h).
- Contrôle des connaissances :
 - Comptes rendus de TPs : 02/11 (TP1 et TP2), 28/12 (TP3 et TP4). Jusqu'à 23h59.
 - Soutenances de TPs : 06/01. Envoi des transparences jusqu'à 23h59 du 05/01.
 - Note des comptes rendus : $N_{CR} = 0.4 \times CR_1 + 0.6 \times CR_2$.
 - Note finale : $0.6 \times N_{CR} + 0.4 \times N_S$.
- Site du cours : https://github.com/felipefr/numerical_analysis.
- Outils à maîtriser :
 - Python et libraries numériques (numpy, matplotlib).
 - Environnements Python : VSCode et/ou Jupyter (e.g. googlecode).
 - Latex : écriture des CRs (Word est fortement déconseillé).
 - Environnements Latex : Texstudio (local) ou Overleaf (remote).

- Presentations : Powerpoint, Beamer (Latex), etc.
- Liens utiles:
 - MATLAB-Python conversion
 - Videos cours Marco Picasso

References

[Quarteroni et al.(2007)Quarteroni, Sacco and Saleri] Quarteroni, A., Sacco, R., Saleri, F., 2007. Numerical Mathematics. volume 37 of *Texts in Applied Mathematics*. 2 ed., Springer, New York, NY. doi:10.1007/b98885.

2 Concepts généraux

On va préciser les définitions:

- Consistance (tableau)
- Convergence (tableau)

3 Représentation numérique sur l'ordinateur et erreurs numériques

Nous allons d'abord identifier les types d'erreurs les plus fréquemment rencontrées lors de la simulation numérique d'un problème de physique par ordinateur.

1. Erreurs de modélisation : simplification ou limitations des modèles mathématiques de la physique.
2. Erreurs de représentation sur ordinateur.
3. Erreurs de troncature.

Nous allons maintenant discuter de ces trois types d'erreurs dans un contexte simple.

Exemple 1 (Pendule avec frottement). *Le système du pendule vertical est défini par la masse m , ici supposée ponctuelle, la longueur du fil ℓ , et l'angle $\theta(t)$ par rapport à l'axe vertical, qui varie en fonction du temps. La force gravitationnelle, avec l'accélération de la pesanteur g , agit sur la particule verticalement vers le bas, et la force $F_f(t)$ due au frottement est contraire à son mouvement. L'application du principe fondamental de la mécanique à ce système donne l'équation différentielle du mouvement ci-dessous :*

$$m\ell\ddot{\theta}(t) + mg \sin \theta(t) + F_f(t) = 0 \quad (1)$$

Le fait de négliger d'autres forces possibles, comme la poussée d'Archimède que l'on considérerait suffisamment importante, constitue déjà une forme d'erreur de modélisation. Si l'on néglige également la force de frottement, étant donné que sa modélisation est assez complexe (et même très inexacte) et que ses effets sont faibles pour de petites vitesses, on augmente encore l'erreur dite de modélisation.

Ensuite, une approximation très souvent considérée est $\sin \theta(t) \approx \theta(t)$, ce qui permet de revenir à l'EDO linéaire. Cela n'est rien d'autre que la troncature au premier ordre de la série de Taylor pour le sinus¹. On a donc ici une erreur de troncature. L'équation simplifiée finale est $m\ell\ddot{\theta}(t) + mg \sin \theta(t) = 0$.

Finalement, l'application d'une méthode numérique pour les EDOs, comme la méthode d'Euler (voir Section 6), qui est une approximation de premier ordre de Taylor, ajoute une nouvelle couche d'erreur de troncature. L'implémentation de ce dernier algorithme sur l'ordinateur va encore ajouter des erreurs liées à la représentation numérique, qui n'est pas exacte pour la plupart des nombres. Ces erreurs s'accumuleront au fur et à mesure que les itérations temporelles évoluent.

¹Il faut se rappeler qu'autour de $\theta_0 = 0$, la série de Taylor du sinus est telle que $\sin \theta = \theta - \frac{1}{3!}\theta^3 + \frac{1}{5!}\theta^5 + \dots$

3.1 Représentation en nombre flottant

Avant d'introduire la représentation en virgule flottante, il est utile de revoir la représentation d'un nombre naturel quelconque. Supposons que dans une base $b \in \mathbb{N}/\{0, 1\}$, un entier $N \in \mathbb{N}$ puisse être représenté par n chiffres :

$$(N)_b = (a_{n-1}a_{n-2} \dots a_1a_0)_b = a_{n-1}b^{n-1} + a_{n-2}b^{n-2} + \dots a_1b + a_0, \quad (2)$$

où les chiffres naturels a_i satisfont $0 \leq a_i < b$ pour tout $i = 0, \dots, n-2$, et $0 < a_{n-1} < b$. La base la plus couramment utilisée est $b = 10$, par exemple, $(926)_{10} = 9 \times 100 + 2 \times 10 + 6$. Il est toujours possible de convertir un nombre d'une base à une autre en examinant les restes des divisions successives de ce nombre par la base. Une base particulièrement importante pour l'informatique est la base binaire ($b = 2$), sur laquelle la plupart des ordinateurs sont construits.

Pour les nombres réels, la représentation reste similaire:

$$x = \pm m \times b^\ell, \quad (3)$$

$$\ell \in \mathbb{Z}, b \in \mathbb{N}/\{0, 1\}, m = 0, d_1d_2 \dots d_n, \quad ,$$

$$\text{tel que } 1 \leq d_i \leq b-1 \text{ et } 0 \leq d_i \leq b-1, \text{ pour } i = 2, \dots, n. \quad (4)$$

En particulier, la partie m est appelée la mantisse. Il est crucial de noter qu'en raison de la précision finie, seule une quantité dénombrable de réels peut être représentée. Si l'on ajoute des bornes inférieures ($\ell_{\min}$) et supérieures ($\ell_{\max}$) pour limiter l'exposant ℓ , la quantité de nombres représentables devient finie. Nous désignons par $\mathbb{F}(n, b, \ell_{\min}, \ell_{\max})$ l'ensemble des nombres flottants en précision finie. Bien évidemment, $\mathbb{F}(n, b, \ell_{\min}, \ell_{\max}) \subset [x_{\min}, x_{\max}] \subset \mathbb{R}$, où $x_{\min}$ et $x_{\max}$ sont facilement déterminés à partir de la définition de $\mathbb{F}$.

Il est important de gérer les exceptions notables qui sortent du cadre de la définition de $\mathbb{F}$. Par exemple, on constate facilement que $0 \notin \mathbb{F}$. Il est également utile de traiter les réels en dehors de l'intervalle admissible ou les résultats de la division par zéro, ce qui définit $\pm\text{infty}$, respectivement au-dessus et en-dessous des bornes. Enfin, un échec ou un résultat d'opération indéterminé, tel que $\frac{0}{0}$, est défini comme NaN (*Not a Number*). Dans la Section 3.1.1, nous précisons la représentation de ces exceptions selon la norme IEEE-754.

Définition 1 (Précision machine). Soit $x \in [x_{\min}, x_{\max}] \subset \mathbb{R}$ et $\text{fl}(x) \in \mathbb{F}$. On définit la précision de la machine ϵ_m comme la plus grande erreur relative possible entre x et $\text{fl}(x)$, c'est-à-dire, $\epsilon_m = \sup_x \frac{|x - \text{fl}(x)|}{|x|}$.

Proposition 1. La précision machine pour $\mathbb{F} = \mathbb{F}(n, b, \ell_{\min}, \ell_{\max})$ est $\epsilon_m = b^{1-n}$ pour $\text{fl} = \text{tronc}$ et $\epsilon_m = \frac{b^{1-n}}{2}$ pour $\text{fl} = \text{arr}$.

Proof. Soit $x = (0, d_1d_2 \dots d_nd_{n+1} \dots) \times b^\ell$. Pour $\text{fl} = \text{tronc}$, on a $\text{fl}(x) = (0, d_1d_2 \dots d_n) \times b^\ell$. On remarque aussi que $|x| \geq (0, 10 \dots) \times b^\ell = b^{\ell-1}$. Ainsi :

$$\frac{|x - \text{fl}(x)|}{|x|} = \frac{(0, \overbrace{0 \dots 0}^{n \times} d_{n+1}d_{n+2} \dots) \times b^\ell}{|x|} \leq \frac{(0, \overbrace{0 \dots 0}^{n-1 \text{ fois}} 10 \dots) \times b^\ell}{b^{\ell-1}} = \frac{b^{\ell-n}}{b^{\ell-1}} = b^{1-n} \quad (5)$$

□

Points importants :

- Nombres les plus grands et les plus petits représentables.
- À l'intérieur de l'intervalle, seul un nombre fini de nombres sont représentables.

3.1.1 Norme IEEE-754

Analysons d'abord le cas d'un système binaire ($b = 2$) avec 32 bits. Le premier bit indique le signe $\pm$. Du deuxième au neuvième bit (8 bits), l'exposant est décrit. Enfin, du dixième au dernier bit (23 bits) sont réservés à la mantisse. Étant donné que le premier bit de la mantisse est différent de 0, il est forcément égal à 1, ce qui implique $n = 24$ ($= 23 + 1$). Avec 8 bits dans l'exposant, la norme admet donc $\ell_{min} = -127$ à $\ell_{max} = 128$ (256 entiers au total), en d'autres termes, l'entier non signé est décalé de 127 vers le bas. Ce qui donne :

$$x_{min} = -0,10 \dots 0 \times 2^{-127} = 3,4 \times 10^{38} \quad (6)$$

$$x_{max} = 0,11 \dots 1 \times 2^{128} = 3,4 \times 10^{38} \quad (7)$$

$$\epsilon_m = 2^{1-24} = 0,119 \times 10^{-6} \quad (8)$$

3.2 Arithmétique flottante en pratique

Points importants :

- L'ordre des opérations est crucial.
- Opérations à risque :
 1. Somme de nombres d'ordres de grandeur différents.
 2. Soustraction de nombres proches.

Exemple à compléter avec l'exécution du script qui exemplifie numériquement les erreurs : https://github.com/felipefr/numerical_analysis/blob/main/tp0/intro_python_basique.ipynb.

4 Regression et Interpolation

Des données expérimentales arrivent normalement en forme échantillonné, c'est-à-dire, on ne connaît qu'un ensemble fini des points, même si la variable d'intérêt est de nature continue (e.g. la température sur Paris au long d'une journée, contrainte et déformation dans essai de traction, etc). On a deux famille d'approches pour approcher ces données afin d'avoir une représentation continue:

1. Regression: On cherche une courbe qui passe le plus proche de l'ensemble de données expérimentale.
2. Interpolation: On cherche une courbe qui passe exactement parmi tout les points de l'ensemble de données.

Soit l'ensemble de données $\{(x_i, y_i) \in \mathbb{R}^2, i = 0, \dots, n\}$ avec $(n+1)$ points. Plus précisément, pour l'approche dite d'interpolation, on cherche à trouver une fonction $f : \mathbb{R} \rightarrow \mathbb{R}$ telle que $f(x_i) = y_i, \forall i = 0, \dots, n$.

On s'intéresse surtout à l'interpolation polynomiale, et pour cela, c'est important de définir quelques notations. On pose $p_n : \mathbb{R} \rightarrow \mathbb{R}$, un polynôme d'ordre n , comme étant une fonction de la forme (pour $a_n \neq 0$) :

$$p_n(x) = a_0 + a_1x + \dots + a_nx^n. \quad (9)$$

4.1 La méthode de moindres carrés

On résout le problème suivant pour $m < n$

$$\min_{\mathbf{a} \in \mathbb{R}^{m+1}} \sum_{i=1}^n |y_i - p_m(x)|^2, \quad (10)$$

où m est le degré du polynôme cherché.

La solution de ce problème consiste à résoudre le système linéaire suivant (preuve au tableau)

$$(\mathbf{V}^T \mathbf{V}) \mathbf{a} = \mathbf{V}^T \mathbf{Y}, \quad (11)$$

avec la matrice $\mathbf{V} \in \mathbb{R}^{(n+1) \times (m+1)}$ (Vandermonde) comme ci-dessous

$$\mathbf{V} = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^m \\ 1 & x_1 & x_1^2 & \dots & x_1^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^m \end{bmatrix}. \quad (12)$$

4.2 Intepolation polynomiale

Avant de commencer, rappelons quelques résultats:

Proposition 2. p_n possède n racines en tenant compte de leurs multiplicités.

Corollaire 1. Étant donné $(n+1)$ points de collocation, il existe un unique polynôme $p_n(x)$.

Proof. Par l'absurde, soient $p_n(x)$ et $q_n(x)$ deux polynômes différents passant par les mêmes points de collocation. On construit :

$$P_n(x) = p_n(x) - q_n(x).$$

On a donc :

$$P_n(x_i) = p_n(x_i) - q_n(x_i) = 0, i = 1, \dots, n.$$

$P_n(x)$ aura donc $(n+1)$ racines, ce qui implique qu'il n'est pas un polynôme de degré n . $\square$

Inconvénients : indépendamment de la méthode, le polynôme recherché est le même.

4.2.1 Méthode 1: Matrice de Vandermonde

On impose $p_n(x_i) = y_i$ pour tout $i = 0, \dots, n$, ce qui revient à la résolution du système linéaire suivant :

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}. \quad (13)$$

Problèmes : mauvais conditionnement et coût de résolution élevé pour un grand n .

4.2.2 Méthode 2: Interpolation de Lagrange

Soit le polynôme :

$$L_i(x) = \frac{\prod_{j=0, j \neq i}^n (x - x_j)}{\prod_{j=0, j \neq i}^n (x_i - x_j)} \quad (14)$$

On peut facilement vérifier que :

$$L_i(x_j) = \delta_{ij} \quad (15)$$

Le polynôme recherché est donc :

$$p_n(x) = \sum_{i=0}^n L_i(x) y_i. \quad (16)$$

Avantages : pas de système linéaire à résoudre.

Inconvénients : reconstruction des polynômes à chaque fois qu'un nouveau point est ajouté.

4.2.3 Méthode 3: Différences divisées de Newton

On réécrit de manière équivalente le polynôme recherché comme :

$$p_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0) \dots (x - x_{n-1}).$$

Ce polynôme satisfait trivialement que :

$$y_0 = p_n(x_0) = a_0, \quad (17)$$

$$y_1 = p_n(x_1) = a_0 + a_1(x_1 - x_0) \Rightarrow a_1 = \frac{y_1 - y_0}{x_1 - x_0}. \quad (18)$$

Définition 2. *Différences divisées de Newton d'ordre 1 :*

$$y[x_i, x_{i+1}] = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}. \quad (19)$$

Définition 3. *Différences divisées de Newton d'ordre 2 :*

$$y[x_i, x_{i+1}, x_{i+2}] = \frac{y[x_{i+1}, x_{i+2}] - y[x_i, x_{i+1}]}{x_{i+2} - x_i}. \quad (20)$$

Définition 4. *Différences divisées de Newton d'ordre n :*

$$y[x_0, x_1, \dots, x_n] = \frac{y[x_0, x_1, \dots, x_{n-1}] - y[x_1, x_2, \dots, x_n]}{x_n - x_0}. \quad (21)$$

Proposition 3. *Pour $m \leq n$:*

$$p_m(x) = p_{m-1}(x) + a_m(x - x_0) \dots (x - x_{m-1}). \quad (22)$$

avec p_{m-1} passant par m points de collocation de x_0 à x_{m-1} et $a_m = y[x_0, \dots, x_m]$. Le polynôme recherché correspond à $m = n$.

4.2.4 Erreur d'interpolation

Soit $f : \mathbb{R} \rightarrow \mathbb{R}$, que l'on veut approcher par le polynôme p_n passant par les points $(x_i, y_i = f(x_i))$. On définit l'erreur comme :

$$e_n(x) = f(x) - p_n(x) \quad (23)$$

Théorème 1. *Soit $x_0 < x_1 < \dots < x_n$ et $f : [x_0, x_n] \rightarrow \mathbb{R}$ une fonction $(n+1)$ -fois dérivable dans $]x_0, x_n[$. Donc $\forall x \in [x_0, x_n]$, $\exists \xi(x) \in]x_0, x_n[$ tel que :*

$$e_n(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!} (x - x_0)(x - x_1) \dots (x - x_n) \quad (24)$$

Proof. Au tableau. □

Corollaire 2. *Si la division est équidistante $s := (x - x_0)/h$, avec $h = (x_n - x_0)/n$, on a :*

$$e_n(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!} s(s-1) \dots (s-n) h^{n+1}. \quad (25)$$

Une autre caractérisation intéressante est la constante de Lebesgue, définie comme :

$$\Lambda_n = \sup_{x \in [a,b]} \sum_{i=0}^n |L_i(x)| \approx \frac{2^{n+1}}{en \log n}, \quad (26)$$

qui correspond à la constante multiplicatrice de la majoration suivante :

$$|p_n(x)| = \left| \sum_i L_i(x) f(x_i) \right| \leq \left(\sum_i |L_i(x)| \right) \|f\|_\infty \quad (27)$$

Observations : l'erreur est nulle aux points de collocation. Question : Pensez-vous qu'en augmentant n on diminue l'erreur ? Contre-exemple : $f(x) = \frac{1}{1+x^2}$.

4.2.5 Interpolation par intervalles

On admet $x_0 < x_1 < \dots < x_n$ et $h = \max_{0 \leq i \leq n-1} |x_{i+1} - x_i|$.

Théorème 2. Soit $f : [a, b] \rightarrow \mathbb{R}$ une fonction $(n+1)$ -fois continûment dérivable. Alors :

$$\max_{x \in [a,b]} |f(x) - f_n(x)| \leq Ch^{n+1} \quad (28)$$

Proof. Au tableau. □

Définition 5 (Spline). La fonction $s_j^k : [a, b] \rightarrow \mathbb{R}$ est une spline de degré k relative au nœud x_j si :

1. $s_j^k|_{[x_j, x_{j+1}]} \in P_k$, $\forall j = 0, \dots, n-1$ et
2. $s_j^k \in C^{k-1}[a, b]$.

4.2.6 Construction des splines cubiques

Ici, $k = 3$ sera omis. On aura $s_{i-1} \in P^3([x_{i-1}, x_i])$ et $s_i \in P^3([x_i, x_{i+1}])$, et on cherche :

$$s_{i-1}(x_i) = s_i(x_i) = y_i, \quad (29)$$

$$s'_{i-1}(x_i) = s'_i(x_i) \quad (30)$$

$$s''_{i-1}(x_i) = s''_i(x_i) \quad (31)$$

On définit :

$$s''_i(x) = M_i \frac{x_{i+1} - x}{h_i} + M_{i+1} \frac{x - x_i}{h_i}, \quad (32)$$

ce qui satisfait par construction la dernière contrainte que $s''_i(x_i) = M_i = s_{i-1}(x_i)$.

$$s_i(x) = M_i \frac{(x_{i+1} - x)^3}{6h_i} + M_{i+1} \frac{(x - x_i)^3}{6h_i} + C_i(x - x_i) + \tilde{C}_i \quad (33)$$

En imposant $s_i(x_i) = y_i$, on obtient :

$$\tilde{C}_i = y_i - M_i \frac{h_i^2}{6} \quad (34)$$

En imposant $s_i(x_{i+1}) = y_{i+1}$, on obtient :

$$C_i = \frac{y_{i+1} - \tilde{C}_i}{h_i} - M_{i+1} \frac{h_i}{6} = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{6} (M_{i+1} - M_i). \quad (35)$$

On s'intéresse aussi aux dérivées premières en x_i :

$$s'_i(x_i) = -M_i \frac{h_i}{2} + \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{6}(M_{i+1} - M_i), \quad (36)$$

$$s'_{i-1}(x_i) = M_i \frac{h_{i-1}}{2} + \frac{y_i - y_{i-1}}{h_{i-1}} - \frac{h_{i-1}}{6}(M_i - M_{i-1}) \quad (37)$$

En imposant $s'_i(x_i) = s'_{i-1}(x_i)$, on obtient le système pour $i = 1, \dots, n-1$:

$$\frac{h_{i-1}}{6}M_{i-1} + \frac{h_{i-1} + h_i}{3}M_i + \frac{h_i}{6}M_{i+1} = \frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}. \quad (38)$$

Nous avons donc $n+1$ variables et $(n-1)$ équations. Le choix $M_0 = M_n = 0$ (dit naturel) est nécessaire pour obtenir l'unicité de la solution.

5 Intégration Numérique

5.1 Newton-Cotes

La méthode de Newton-Cotes consiste à approcher l'intégrale de f par l'intégrale de son polynôme interpolant. Ainsi :

$$\int_{x_0}^{x_n} f(x)dx = \int_{x_0}^{x_n} p_n(x)dx + \int_{x_0}^{x_n} e_n(x)dx \approx \int_{x_0}^{x_n} p_n(x)dx \quad (39)$$

5.1.1 Méthode des trapèzes

Pour $n = 1$:

$$\begin{aligned} \int_{x_0}^{x_1} p_1(x) &= \int_{x_0}^{x_1} y_0 + f[x_0, x_1](x - x_0) \\ &= y_0 h + \frac{(y_1 - y_0)h}{2} = \frac{h}{2}(y_0 + y_1). \end{aligned} \quad (40)$$

Pour l'erreur :

$$\begin{aligned} E_1 &= \int_{x_0}^{x_1} e_1(x) = \int_{x_0}^{x_1} \frac{1}{2} f''(\xi(x))(x - x_0)(x - x_1) \\ &= \frac{1}{2} f''(\eta) h^3 \int_0^1 s(s-1)ds = -\frac{1}{12} f''(\eta) h^3 \end{aligned} \quad (41)$$

où le Théorème 3 a été utilisé étant donné que le polynôme $(x - x_0)(x - x_1)$ ne change pas de signe dans $[x_0, x_1]$.

5.1.2 Méthode de Simpson

Pour $n = 2$:

$$\begin{aligned} \int_{x_0}^{x_2} p_2(x) &= \int_{x_0}^{x_2} f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ &= \int_0^2 (f[x_0] + f[x_0, x_1]sh + f[x_0, x_1, x_2]s(s-1)h^2)hds \\ &= h \left(2y_0 + 2\frac{y_1 - y_0}{h}h + \frac{2}{3}\frac{y_2 - 2y_1 + y_0}{2h^2}h^2 \right) \\ &= \frac{h}{3}(y_0 + 4y_1 + y_2). \end{aligned} \quad (42)$$

L'analyse de l'erreur est plus subtile car la forme :

$$E_2 = \int_{x_0}^{x_2} e_2(x) = \int_{x_0}^{x_2} \frac{1}{3!} f'''(\xi(x))(x-x_0)(x-x_1)(x-x_2), \quad (43)$$

empêche l'utilisation du Théorème 3, étant donné que $(x-x_0)(x-x_1)(x-x_2)$ change de signe dans $[x_0, x_2]$. On va donc construire :

$$p_3(x) = p_2(x) + (f(x_3) - p_2(x_3)) \frac{(x-x_0)(x-x_1)(x-x_2)}{(x_3-x_0)(x_3-x_1)(x_3-x_2)}, \quad (44)$$

passant par un point x_3 (non utilisé pour l'intégration) de telle sorte que $p_3(x_3) = f(x_3)$ et $p_3(x_i) = p_2(x_i) = f(x_i)$ pour $i = 0, \dots, 2$. En vérifiant que $\int_0^2 s(s-1)(s-2)ds = 0$, on constate que :

$$\int_{x_0}^{x_2} p_2(x)dx = \int_{x_0}^{x_2} p_3(x)dx. \quad (45)$$

Ceci étant, on a :

$$E_3 = \int_{x_0}^{x_2} e_3(x) = \int_{x_0}^{x_2} \frac{1}{4!} f^{(4)}(\xi(x))(x-x_0)(x-x_1)(x-x_2)(x-x_3). \quad (46)$$

Le choix $x_3 = x_1$ garantit que $(x-x_0)(x-x_1)^2(x-x_2)$ reste positif dans $[x_0, x_2]$. Alors :

$$E_3 = \int_{x_0}^{x_2} \frac{1}{4!} f^{(4)}(\xi(x))(x-x_0)(x-x_1)^2(x-x_2) \quad (47)$$

$$= \frac{h^5}{4!} f^{(4)}(\eta) \int_0^2 s(s-1)^2(s-2) = \frac{-4}{15} \frac{h^5}{24} f^{(4)}(\eta) = -\frac{h^5}{90} f^{(4)}(\eta) \quad (48)$$

Obs : La méthode de Taylor ne donne pas la constante correctement.

5.2 Formules composées

Pour la méthode des trapèzes :

$$\begin{aligned} \int_a^b f(x)dx &= \sum_{i=0}^{m-1} \int_{x_i}^{x_{i+1}} f(x)dx = \sum_{i=0}^{m-1} \int_{x_i}^{x_{i+1}} p_n(x)dx + \sum_{i=0}^{m-1} \int_{x_i}^{x_{i+1}} e_n(x)dx \\ &= I_n^m(f) - \sum_{i=0}^{m-1} \frac{1}{12} f''(\eta) h^3, \end{aligned} \quad (49)$$

$$\Rightarrow E_{n,m} \leq \frac{m}{12} \sup_{x \in [a,b]} |f''(x)| h^3 = Ch^2 \quad (50)$$

$$\text{avec } C = \frac{(b-a)}{12} \sup_{x \in [a,b]} |f''(x)|$$

Pour la méthode de Simpson :

$$\Rightarrow E_{n,m} \leq \frac{m}{90} \sup_{x \in [a,b]} |f^{(4)}(x)| h^5 = \frac{b-a}{2h \times 90} \sup_{x \in [a,b]} |f^{(4)}(x)| h^5 = Ch^4 \quad (51)$$

$$\text{avec } C = \frac{(b-a)}{180} \sup_{x \in [a,b]} |f^{(4)}(x)|$$

5.3 Quadrature de Gauss (approche direct)

Les méthodes de Newton-Côtes supposent l'évaluation des intégrales aux points extrêmes des intervalles qui définissent des partitions. Dans la quadrature gaussienne, ces points se trouvent à l'intérieur de l'intervalle. L'accès à un plus grand nombre de degrés de liberté dans le schéma d'intégration permettra donc une amélioration du rapport degré d'exactitude du schéma (degré maximal du polynôme dont la quadrature est exacte) par rapport au nombre de points utilisés.

$$\int_{-1}^1 f(x)dx = \sum_{i=1}^{N_q} \omega_i f(x_i) + e_{N_q} \quad (52)$$

Notre objectif est de trouver ω_i et x_i pour $i = 1, \dots, N_q$ de telle sorte qu'ils éliminent l'erreur d'intégration pour certaines fonctions de base ϕ_j , $j = 1, \dots, m$. C'est-à-dire :

$$\int_{-1}^1 \phi_j(x)dx = \sum_{i=1}^{N_q} \omega_i \phi_j(x_i). \quad (53)$$

Si $\phi_j(x) = x^{j-1}$ (la base canonique des polynômes), $m - 1$ est le degré maximal des polynômes à intégrer de façon exacte.

Exemple 2 (Schéma à 1 point (QG1)). Prenons $N_q = 1$ et $m = 2$. Donc :

$$\begin{cases} \int_{-1}^1 1dt = 2 & = \omega_1, \\ \int_{-1}^1 tdt = 0 & = \omega_1 t_1 \end{cases} \Rightarrow \omega_1 = 2, t_1 = 0. \quad (54)$$

Exemple 3 (Schéma à 2 points (QG2)). Prenons $N_q = 2$ et $m = 4$. Donc :

$$\begin{cases} \int_{-1}^1 1dt = 2 & = \omega_1 + \omega_2, \\ \int_{-1}^1 tdt = 0 & = \omega_1 t_1 + \omega_2 t_2, \\ \int_{-1}^1 t^2dt = 2/3 & = \omega_1 t_1^2 + \omega_2 t_2^2, \\ \int_{-1}^1 t^3dt = 0 & = \omega_1 t_1^3 + \omega_2 t_2^3 \end{cases} \Rightarrow \omega_1 = \omega_2 = 1, t_1 = +\sqrt{\frac{1}{3}}, t_2 = -\sqrt{\frac{1}{3}}. \quad (55)$$

5.3.1 Transformation depuis un élément de référence

On définit $t \mapsto \varphi(t) := \frac{(a+b)}{2} + \frac{(b-a)}{2}t$, de telle sorte que $x = \varphi(t)$, pour $t \in [-1, 1]$. Alors :

$$\int_a^b f(x)dx = \int_{\varphi(-1)}^{\varphi(1)} f(x)dx = \frac{(b-a)}{2} \int_{-1}^1 (f \circ \varphi)(t) \varphi'(t)dt \approx \sum_{i=1}^{N_q} \omega_i (f \circ \varphi)(t_i). \quad (56)$$

5.4 Quadrature de Gauss (approche par polynômes orthogonaux)

5.4.1 Procédure d'orthogonalisation de Gram-Schmidt

Soit le produit interne $(\bullet, \star)$ (e.g. $\int_{-1}^1 w(x)f(x)g(x)dx$), et les polynômes $\{\phi_0, \phi_1, \dots, \phi_n\}$, avec $\phi_k \in \mathbb{P}^n, \forall i = 0, \dots, n$, formant une base de $\mathbb{P}^n$. La base $\{\varphi_1, \varphi_2, \dots, \varphi_n\}$ construit comme suit est orthonormale:

$$\begin{cases} \tilde{\varphi}_k(x) & = \phi_k(x) - \sum_{i=0}^{k-1} (\varphi_i, \phi_k), \\ \varphi_k(x) & = \frac{1}{(\tilde{\varphi}_k, \tilde{\varphi}_k)} \tilde{\varphi}_k(x) \end{cases}, \quad \forall k = 0, \dots, n. \quad (57)$$

5.5 Intégration en 2 dimensions (application aux éléments finis)

A compléter au tableau.

n_p	$B_{0,n}$	$B_{1,n}$	$B_{2,n}$	$B_{3,n}$	$B_{4,n}$
2	1/2	1/2			
3	1/6	4/6	1/6		
4	1/8	3/8	3/8	1/8	
5	7/90	32/90	12/90	32/90	7/90

Table 1: Newton-Cotes weights.

n_p	w_i	ξ_i
1	2	0
2	1, 1	$-1/\sqrt{3}, 1/\sqrt{3}$
3	5/9, 8/9, 5/9	$-\sqrt{3/5}, 0, \sqrt{3/5}$

Table 2: Gauss weights.

5.6 Résumé

Note : $i = 0..(n_p - 1)$.

6 Introduction aux méthodes de résolution numérique des équations différentielles ordinaires

On pose le problème de Cauchy de trouver $u : \mathbb{R}^+ \rightarrow \mathbb{R}$:

$$\begin{cases} u'(t) &= f(t, u(t)), \\ u(t_0) &= y_0 \end{cases} \quad (58)$$

où f est lipschitzienne par hypothèse. Nous allons analyser quelques schémas vis-à-vis de l'erreur de troncature locale définie par :

$$\tau = \frac{u(t_{n+1}) - u(t_n)}{h} - \Phi(t_n, y(t_n)), \quad (59)$$

où Φ est la fonction qui définit la méthode. La construction des méthodes se fait en utilisant les méthodes d'intégration numériques déjà étudiées.

6.1 Méthode d'Euler explicite

En appliquant le théorème fondamental de l'analyse et l'approximation de degré zéro autour de t_n , on a :

$$\begin{aligned} \int_{t_n}^{t_{n+1}} u'(t) dt &= u(t_{n+1}) - u(t_n) = \int_{t_n}^{t_{n+1}} f(t, u(t)) dt \\ &\approx \int_{t_n}^{t_{n+1}} p_0(t) dt = \int_{t_n}^{t_{n+1}} f(t_n, u_n) dt = f(t_n, u_n) h \end{aligned} \quad (60)$$

La méthode d'Euler est donc :

$$u_{n+1} = u_n + f(t_n, u_n)h := \Phi^{EE}(t_n, u_n). \quad (61)$$

6.2 Méthode d'Euler implicite

De façon analogue, mais en utilisant l'approximation de degré 0 autour de t_{n+1} , on a :

$$u_{n+1} = u_n + f(t_{n+1}, u_{n+1})h := \Phi^{EI}(t_{n+1}, u_{n+1}). \quad (62)$$

6.3 Méthode Adams-Bashford

Pour appliquer l'approximation de degré 1 qui soit explicite, il faut utiliser un polynôme interpolant des instants t_{n-1} et t_n , même si l'intégrale est réalisée entre t_n et t_{n+1} . Alors :

$$\begin{aligned}\Phi^{AB1}(t_{n-1}, t_n, u_n, u_{n-1}) &= \int_{t_n}^{t_{n+1}} p_1(t) dt = \int_{t_n}^{t_{n+1}} f_n + f[t_n, t_{n-1}](t - t_n) dt = \\ &= f_n h + \frac{f_{n-1} - f_n}{-h} \frac{h^2}{2} = \frac{h}{2} (3f_n - f_{n-1}).\end{aligned}\quad (63)$$

Obs 1 : il faut noter que $x_0 = t_n$ et $x_1 = t_{n-1}$ avec les notations précédentes. Ce qui fait que $x_1 - x_0 = -h$.

Obs 2 : La méthode d'Euler explicite n'est que l'AB0.

Exercice 1. Montrer que :

$$\Phi^{AB2}(t_{n-2}, t_{n-1}, t_n, u_n, u_{n-1}, u_{n-2}) = \frac{h}{12} (23f_n - 16f_{n-1} + 5f_{n-2})$$

6.4 Méthode Adams-Moulton

De façon analogue, mais en utilisant un polynôme interpolant des instants t_n et t_{n+1} , on obtient une méthode implicite :

$$\begin{aligned}\Phi^{AM1}(t_n, t_{n+1}, u_n, u_{n+1}) &= \int_{t_n}^{t_{n+1}} p_1(t) dt = \int_{t_n}^{t_{n+1}} f_{n+1} + f[t_{n+1}, t_n](t - t_{n+1}) dt = \\ &= f_{n+1} h + \frac{f_n - f_{n+1}}{-h} \frac{(-h^2)}{2} = \frac{h}{2} (f_n + f_{n+1}).\end{aligned}\quad (64)$$

On note que:

- La méthode AM1 correspond à la règle du trapèze d'intégration, plus connu comme Crank-Nicholson dans ce contexte.
- La méthode l'AM0 correspond à la méthode d'Euler implicite (déjà étudié).

Exemple 4 (Problème de Crank-Nicholson à un problème non-linéaire). Nous allons appliquer la méthode de Crank-Nicolson à l'équation différentielle ordinaire non linéaire suivante $u'(t) = au^2$ avec $a > 0$ une constante donné et $u(t_0) = u_0$, ce que résulte

$$u_{n+1} - u_n = \frac{ah}{2} (u_n^2 + u_{n+1}^2).$$

En réarrangeant les termes on abouti de la forme $Ax^2 + Bx + C = 0$ (quadratique) à résoudre

$$\underbrace{\frac{ah}{2} u_{n+1}^2}_{:=A} + \underbrace{(-1) u_{n+1}}_{:=B} + \underbrace{\left(u_n + \frac{ah}{2} u_n^2\right)}_{:=C} = 0,$$

dont la solution est

$$u_{n+1} = \frac{1 \pm \sqrt{1 - 2ahu_n - (ah)^2 u_n^2}}{ah}.$$

Il est nécessaire de choisir la solution (le signe $\pm$) qui correspond au comportement physique de la solution ou qui est numériquement stable. Pour des problèmes plus complexes, la résolution de cette équation non linéaire à chaque pas de temps est souvent effectuée par des méthodes itératives, telles que la méthode de Newton-Raphson (pas étudié en cours, mais dans l'annexe ??).

Exercice 2. Montrer que :

$$\Phi^{AM2}(t_{n-1}, t_n, t_{n+1}, u_{n-1}, u_n, u_{n+1}) = \frac{h}{12}(5f_{n+1} + 8f_n - f_{n-1})$$

$$\Phi^{AM3}(t_{n-2}, \dots, t_{n+1}, u_{n-2}, \dots, u_{n+1}) = \frac{h}{24}(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2})$$

Exercice 3 (Modèle viscoélastique 1D - Kelvin-Voigt). Un matériau viscoélastique de type Kelvin-Voigt en 1D est constitué d'un ressort (représentant l'élasticité) et d'un amortisseur (représentant la viscosité) connectés en parallèle. La contrainte totale $\sigma(t)$ dans le matériau est la somme des contraintes dans le ressort et dans l'amortisseur. Si E est le module d'Young du ressort et η est le coefficient de viscosité de l'amortisseur, et si $\varepsilon(t)$ est la déformation, alors la contrainte est donné par

$$\sigma(t) = E\varepsilon(t) + \eta\dot{\varepsilon}(t). \quad (65)$$

On considère le un cas où $\varepsilon(t_0) = \varepsilon_0$ et la contrainte $\sigma(t)$ est une fonction du temps connue: i) constante $\sigma(t) = \sigma_0$ et ii) sinusoïdale $\sigma(t) = \sigma_0 \sin \omega t$. Après réarranger le problème pour la forme $u'(t) = f(t, u(t))$, appliquer la méthode d'Euler implicite et Crank-Nicholson. Pour l'implémentation numérique, considérer des valeurs raisonnables pour les constantes (e.g. par exemple chercher sur internet les valeurs de E et η pour le caoutchouc.).

6.5 Méthodes de la famille de Runge-Kutta

Les méthodes à un pas vues précédemment sont limitées à l'ordre d'approximation 1. Les méthodes multi-pas ont pour inconvénients leur démarrage et la consommation de mémoire. L'idée derrière les méthodes de Runge-Kutta est d'utiliser des prédictions pour les valeurs à l'instant actuel, puis de les corriger en préservant la consistance. Inspirons-nous des méthodes précédentes et prenons :

$$\Phi(t, u) = b_1 f(t, u) + b_2 f(t + p_1 h, u + h p_2 f(t, u)), \quad (66)$$

notre but est de trouver b_1, b_2, p_1 , et p_2 qui garantissent la consistance. D'un côté, le développement limité autour de (t_n, u_n) donne :

$$\Phi = (b_1 + b_2)f_n + b_2 f_t(t_n, u_n)p_1 h + b_2 f_u(t_n, u_n)f_n h p_2 + \mathcal{O}(h^2). \quad (67)$$

De l'autre côté, pour $u(t_n) = u_n$, on a :

$$\frac{u(t_n + h) - u_n}{h} = f_n + \frac{h}{2}(f_t(t_n, u_n) + f_u(t_n, u_n)f_n) + \mathcal{O}(h^2). \quad (68)$$

Par comparaison :

$$\begin{cases} b_1 + b_2 = 1, \\ b_2 p_1 = \frac{1}{2}, \\ b_2 p_2 = \frac{1}{2} \end{cases} \quad (69)$$

En prenant $\beta = b_2$ comme paramètre libre, la solution est :

$$\begin{cases} b_1 = 1 - \beta \\ p_1 = \frac{1}{2\beta}, \\ p_2 = \frac{1}{2\beta} \end{cases} \quad (70)$$

Si $\beta = 1$, on a la méthode dite Runge-Kutta 1, ou Euler modifiée :

$$\Phi^{RK1}(t_n, u_n; h) = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}f_n\right). \quad (71)$$

Si $\beta = 1/2$, on a la méthode dite Runge-Kutta 1, ou de Heun :

$$\Phi^H(t_n, u_n; h) = \frac{1}{2}(f_n + f(t_n + h, u_n + hf_n)). \quad (72)$$

Ces deux méthodes sont d'ordre 2 par construction. Il est possible de construire des méthodes d'ordre plus élevé en effectuant des prédictions de manière récursive. La vérification de la consistance est, par contre, beaucoup plus fastidieuse. Nous énonçons simplement la méthode de Runge-Kutta d'ordre 4, étant donné sa popularité :

$$\Phi^{RK4}(t_n, u_n; h) = \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4), \quad (73)$$

$$\text{où : } \begin{cases} k_1 &= f_n, \\ k_2 &= f(t_n + \frac{h}{2}, u_n + \frac{h}{2}k_1), \\ k_3 &= f(t_n + \frac{h}{2}, u_n + \frac{h}{2}k_2), \\ k_4 &= f(t_n + h, u_n + hk_3). \end{cases} \quad (74)$$

On peut noter que RK4 peut être interprété comme l'application de la méthode d'intégration de Simpson, où k_2 et k_3 sont des prédictions de la valeur de la fonction dans un pas intermédiaire situé en $t_n + h/2$ (la somme des poids étant 4), et k_4 la prédiction de f_{n+1} .

6.6 Ordre de convergence

Analyse par série de Taylor des schémas précédents (au tableau).

6.7 Stabilité

Ici, nous étudions la stabilité au sens absolu, ce qui correspond à l'étude de l'application des méthodes à l'équation de Dahlquist ci-dessous pour des valeurs négatives de la partie réelle de $\lambda \in \mathbb{C}$:

$$\begin{cases} u'(t) &= \lambda u(t), \\ u(0) &= u_0 \end{cases}. \quad (75)$$

Pour $\lambda = \lambda_R + i\lambda_I$, la solution est $u(t) = u_0 e^{\lambda t} (\cos \lambda_I t + i \sin \lambda_I t)$. Pour qu'un schéma numérique soit considéré stable, il suffit qu'il respecte $|u_{n+1}| \leq |u_n|$.

Exemple 5 (Euler Explicite). *En appliquant le schéma d'Euler explicite, on a :*

$$u_{n+1} = u_n + h\lambda u_n \Rightarrow |u_{n+1}| = |1 + h\lambda| |u_n| \Rightarrow |1 + h\lambda| \leq 1.$$

Pour le cas réel pur, c'est-à-dire $\lambda = \lambda_R < 0$, on a :

$$-1 \leq 1 + h\lambda_R \leq 1 \Rightarrow \frac{-2}{\lambda_R} \geq h \geq 0 \Rightarrow h \leq -\frac{2}{\lambda_R}.$$

Pour éviter des oscillations, il est préférable que :

$$0 \leq 1 + h\lambda_R \leq 1 \Rightarrow \frac{-1}{\lambda_R} \geq h \geq 0 \Rightarrow h \leq -\frac{1}{\lambda_R}.$$

Exercice 4 (Euler Implicite). *Montrer que les schémas Euler implicite et Crank-Nicholson sont inconditionnellement stables.*

Exercice 5 (Runge-Kutta). *Montrer que les conditions de stabilité sont $|Q(\lambda h)| \leq 1$, avec $Q(z) = \sum_{k=0}^p \frac{1}{k!} z^k$, où $p = 2$ pour RK1 et $p = 4$ pour RK4.*

6.8 Applications

6.8.1 Problèmes vectoriels et EDOs d'ordre élevé

Jusqu'au présent, nous n'avons qu'étudié des méthodes numériques pour des EDOs scalaires de première ordre (problème dite de Cauchy). Notre but maintenant c'est de ramener des EDOs d'ordre élevé à des problèmes vectoriels de première ordre et de remarquer que l'extension pour le cas vectoriel est directe. Reprenons l'exemple 1 du pendule, où un problème de deuxième ordre est posé.

On pose $\mathbf{u}(t) = [\theta(t), \dot{\theta}(t)]^T$, naturellement on a

$$\dot{\mathbf{u}}(t) = \begin{bmatrix} \dot{\theta}(t) \\ \ddot{\theta}(t) \end{bmatrix} = \begin{bmatrix} u_2(t) \\ \frac{1}{m\ell}(F_f(t) - mg \sin(u_1(t))) \end{bmatrix} := \mathbf{F}(t, \mathbf{u}(t)). \quad (76)$$

On note que $\mathbf{F} : \mathbb{R}^+ \times \mathbb{R}^2$. L'application, par exemple, de la méthode d'Euler explicite est trivial: étant donné $\mathbf{u}_2 \in \mathbb{R}^2$ on cherche $\mathbf{u}_{n+1} = \mathbf{u}_n + h\mathbf{F}(t_n, \mathbf{u}_n)$.

Exercice 6 (Modèle de la Poutre d'Euler-Bernoulli (Problème à Valeurs Initiales)). *Considérez une poutre d'Euler-Bernoulli de longueur L , de rigidité à la flexion EI constante, soumise à une charge transversale $q(x)$ distribué. L'équation différentielle régissant la déflexion transversale $w(x)$ est donnée par :*

$$EI \frac{d^4 w}{dx^4} = q_0$$

Pour que ce soit un problème à valeurs initiales, nous supposons que nous connaissons la déflexion, la pente, le moment de flexion et l'effort tranchant à l'extrémité $x = 0$. Les conditions sont donc :

$$w(0) = w_0, \frac{dw}{dx}(0) = w'_0, \frac{d^2 w}{dx^2}(0) = w''_0, \frac{d^3 w}{dx^3}(0) = w'''_0$$

Où w_0, w'_0 correspond directement au déflexion et la pente, respectivement, et w''_0, w'''_0 sont reliés au moment et l'effort tranchant avec les formules $M(0) = EIw''_0$ et $V(0) = -EIw'''_0$, respectivement, où $M(0)$ et $V(0)$ peuvent être des valeurs potentiellement donnés ² Définissez clairement vos variables et écrivez le système sous la forme vectorielle $\mathbf{u}'(x) = \mathbf{F}(x, \mathbf{u}(x))$.

6.8.2 Des EDPs (équations différentiels aux dérivée partiels) spatio-temporels

Très souvent en mécanique, des phénomènes sont modélisé par des EDPs spatio-temporel. Prenons par exemple l'équation de la chaleur 1D, on cherche le champs de température $u : [0, 1] \times [0, T] \rightarrow \mathbb{R}$ tel que

$$\begin{cases} \partial_t u(x, t) = k \partial_x^2 u(x, t) & \forall (x, t) \in [0, 1] \times [0, T], \\ u(x, 0) = u_0(x) & \forall x \in [0, 1], \\ u(0, t) = u_G(t) & \forall t \in [0, T], \\ u(1, t) = u_D(t) & \forall t \in [0, T], \end{cases}, \quad (77)$$

où u_0 est un condition initial et u_G et u_D sont de conditions aux limites. Il s'agit donc d'un problème que ne rentre que partielement dans le cadre fixé des pour problèmes aux valeurs initiales (problème de Cauchy). D'autre côté, des méthodes numériques comme les différences finies (MDF) ou d'éléments finis (MEF) (étudiés dans d'autres UEs du M1 et M2) permettent d'adresser des problèmes conditions aux limites. S'on assume l'ansatz $u(x, t) = \sum_{i=1}^{N_h} u_i(t) \phi_i(x)$ les méthodes du type MDF ou MEF permettent de convertir le problème continu (spatielement) dans sa version discretisé (spatielement) de la forme

$$\mathbf{A} \dot{\mathbf{U}}(t) = \mathbf{K} \mathbf{U}(t), \quad (78)$$

²Attention: la forme plus naturelle de formuler ce problème est avec un problème avec conditions aux limites, mais on n'étudie pas les méthodes adaptés pour ce cours.

où $\mathbf{A}, \mathbf{K} \in \mathbb{R}^{N_h \times N_h}$ sont des matrices issues de la discretisation spatiale, et $\mathbf{U} : [0, T] \rightarrow \mathbb{R}^{N_h}$ est un vecteur de degrés de liberté dépendant du temps. On revient donc au cadre des problèmes vectoriels de la section 6.8.1. L'application de la méthode d'Euler explicite et implicite aboutissent respectivement à

6.8.3 Méthodes spécifiques pour des problèmes des EDOs de deuxième ordre

Des EDOs de deuxième ordre arrivent très souvent en mécanique, même dans les cadres plus basiques, comme le principe fondamentale de dynamique (deuxième lois de Newton), e.g. système masse-ressort.

La méthode du saute-mouton (ou *leapfrog method* en anglais) Même qu'il ne soit pas limité à des problèmes de la mécanique, on garde ici la notation utilisé plus souvent, qui est en termes de vitesses et accélérations. On va considérer que l'accélération $a(t)$ n'est fonction que du temps et de la position $x(t)$ d'une particule, comme par exemple dans le cas du système masse-ressort $a(t) = (f(t) - kx(t))/m$, avec $f(t), k$ et m , une force variable dans le temps, la constante de raideur et masse de la particule, respectivement. Etant donné x_n et v_n , l'algorithme suit:

1. Calcul de l'accélération à l'instant t_n : $a_n = a_n(t_n, x_n)$.
2. Mise à jour de la vitesse à mi-pas : $v_{n+1/2} = v_n + a_n \frac{\Delta t}{2}$.
3. Mise à jour de la position au pas complet : $x_{n+1} = x_n + v_{n+1/2}$.
4. Calcul de l'accélération à l'instant t_{n+1} : $a_{n+1} = a_n(t_n + \Delta t, x_{n+1})$.
5. Mise à jour de la vitesse au pas complet (pour le prochain cycle) : $v_{n+1} = v_{n+1/2} + a_{n+1} \frac{\Delta t}{2}$.
6. Pour le prochain pas: $a_n := a_{n+1}, v_n := v_{n+1}, x_n := x_{n+1}, n := n + 1$.

Le méthode *leapfrog* est très appréciée pour sa nature symplectique, ce qui signifie qu'elle conserve bien l'énergie totale du système sur de longues périodes de simulation, contrairement à d'autres méthodes comme Euler explicite qui peuvent introduire des dissipations ou des augmentations d'énergie artificielles. Ce méthode est facilement étendue pour le cadre vectorielle.

References

[Quarteroni et al.(2007)Quarteroni, Sacco and Saleri] Quarteroni, A., Sacco, R., Saleri, F., 2007. Numerical Mathematics. volume 37 of *Texts in Applied Mathematics*. 2 ed., Springer, New York, NY. doi:10.1007/b98885.

A Théorèmes auxiliaires

Théorème 3 (Valeur moyenne des intégrales). *Soit $f_1 \in C^1[a, b]$ et $f_2 \in L^2([a, b])$ qui ne changent pas de signe. Alors $\exists \eta \in [a, b]$ tel que :*

$$\int_a^b f_2(x) f_1(x) dx = f_1(\eta) \int_a^b f_2(x) dx \quad (79)$$

Théorème 4 (Théorème de la série de Taylor (avec reste de Lagrange)). *Soit $f : I \rightarrow \mathbb{R}$ une fonction $(n+1)$ -fois dérivable sur un intervalle ouvert $I \subset \mathbb{R}$, et soit $x_0 \in I$. Alors, $\forall x \in I$, $\exists \xi \in (a, x)$ tel que :*

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n + R_n(x),$$

avec le reste donné par la formule de Lagrange :

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)^{n+1}.$$

B Compléments pour aller au-delà du cours

B.1 Méthode de Newton-Raphson

Dans sa version plus simple (scalaire), pour $f : \mathbb{R} \rightarrow \mathbb{R}$, on cherche $x^* \in \mathbb{R}$ tel que $f(x^*) = 0$. Etant donné $x^{(k)} \in \mathbb{R}$, on cherche donc $dx^{(k)} \in \mathbb{R}$ (par développement limité de Taylor) tel que

$$\begin{aligned} f(x^{(k)}) + dx^{(k)} &= f(x^{(k)}) + f'(x^{(k)})dx^{(k)} + \mathcal{O}([dx^{(k)}]^2) \approx f(x^{(k)}) + f'(x^{(k)})dx^{(k)} = 0, \\ \Rightarrow x^{(k+1)} &:= x^{(k)} + dx^{(k)} = x^{(k)} - [f'(x^{(k)})]^{-1}f(x^{(k)}). \end{aligned} \quad (80)$$

En partant d'un point initial $x^{(0)}$ donné, la méthode suit itérativement jusqu'à atteindre la convergence selon les critères $|f(x^{(k+1)})| < \text{tol}$ ou $\frac{|dx^{(k)}|}{|x^{(k)}|} < \text{tol}$. Il est facilement généralisable pour le cas $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, où l'incrément $d\mathbf{x}^{(k)} \in \mathbb{R}^n$ est donné par la solution du système linéaire $\mathbf{J}(\mathbf{x}^{(k)})d\mathbf{x}^{(k)} = -\mathbf{F}(\mathbf{x}^{(k)})$, où $J_{ij} = \partial_{x_j} F_i, \forall i, j = 1, \dots, n$, est dite la matrice jacobienne.

Sur certaines hypothèses techniques, c'est possible de montrer que la convergence de la méthode est quadratique pour des points assez proches de la solution. Il est nécessaire aussi que la dérivée ne s'annule pas, ou équivalentement, que la matrice jacobienne soit non-singulière, dans une itération donnée. Pour plus de détails, on renvoie le lecteur à l'ouvrage [Quarteroni et al.(2007)Quarteroni, Sacco and Saleri].